تقرير حل تحدي تقني (CTF Write-up)

أيمن عبدالله محمد الدومه

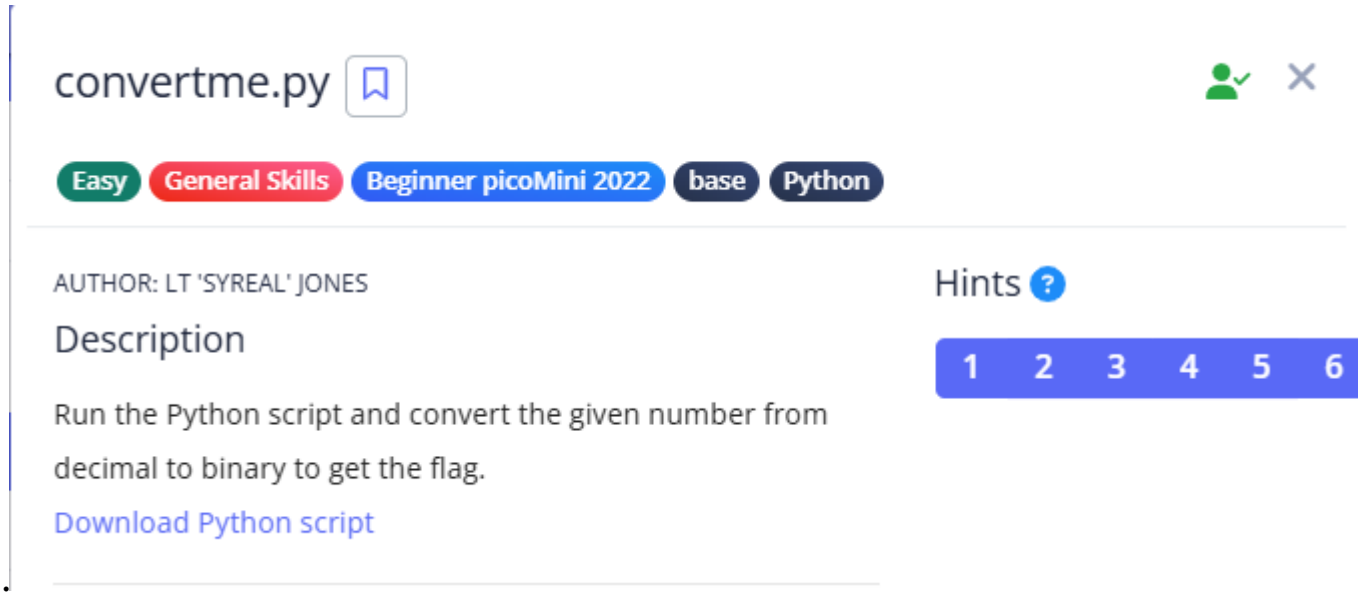
اسم التحدي:convertme.py

المنصة: PicoCTF

الفئة: General Skills

تاريخ الحل: 15 ديسمبر 2025

الحالة: تم الحل(Solved)



الملخص التنفيذي(Executive Summary)

يهدف هذا التحدي إلى اختبار قدرة المشارك على التعامل مع السكريبتات بلغة Python وفهم أنظمة العد الأساسية .(Number Systems) يتطلب التحدي تشغيل سكريبت يقوم بتوليد رقم عشوائي بالنظام العشري(Decimal) ، ويطلب من المستخدم إدخال القيمة المقابلة له بالنظام الثنائي (Binary) للحصول على العلم.(Flag)

المنهجية المتبعة(Methodology)

تم اتباع الخطوات التالية للوصول إلى الحل:

أ. الاستكشاف والتشغيل

1. تم تنزيل ملف السكريبت .convertme.py
2. تم تشغيل السكريبت في بيئة الطرفية (Terminal) باستخدام الأمر python convertme.py
3. قام البرنامج بطباعة رسالة تطلب تحويل الرقم **48**(في هذه المحاولة) من النظام العشري إلى الثنائي.

ب. عملية التحويل(The Conversion Process)

تم استخدام نهج هجين للحل لضمان الدقة والفهم:

1. **التحليل اليدوي :**تم إجراء محاولة أولية لفهم المنطق باستخدام الورقة والقلم (القسمة المتكررة على 2).
2. **التحقق التقني :**تم استخدام أداة عبر الإنترنت (RapidTables) لتسريع العملية والتأكد من صحة الناتج.
   - **المدخل:** (Decimal) 48
   - **الناتج:** (Binary) 110000

ج. استخراج العلم

بعد إدخال القيمة 110000 في البرنامج، قام السكريبت بالتحقق من صحة الإجابة وفك تشفير النص السري وطباعة العلم بنجاح.

العلم (The Flag) :

Plaintext

picoCTF{4ll_y0ur_b4535_762f748e}

التحليل الفني(Technical Deep Dive)

بناءً على سلوك البرنامج، يمكن استنتاج آلية عمل الكود الداخلي كالتالي:

- **توليد الأرقام :**يستخدم السكريبت مكتبة random لتوليد رقم عشوائي في كل مرة يتم تشغيله فيها. هذا يعني أن الإجابة ليست ثابتة.
- **آلية التحقق :**يستخدم السكريبت دالة شرطية تقارن "مدخل المستخدم" مع "ناتج تحويل الرقم العشوائي داخلياً."
  - if user_input == bin(random_num)[2:]:
- **التشفير :**العلم ليس مخزناً كنص واضح (Plaintext) داخل الملف، بل يتم فك تشفيره فقط عند تحقق الشرط الصحيح، مما يمنع الحصول عليه بمجرد قراءة الكود (<code>cat convertme.py</code>) دون تشغيله وفهمه.

الدروس المستفادة والتوصيات(Lessons & Suggestions)

بناءً على تجربتي في هذا التحدي، إليك أهم النقاط المستخلصة للمستقبل:

1. **الفهم قبل الأتمتة :**الحل اليدوي الذي قمت به في البداية هو الأساس المتين. الاعتماد الكلي على الأدوات دون فهم الأساسيات قد يسبب مشاكل في تحديات التشفير المعقدة لاحقاً.
2. **تحسين السرعة:(Optimization)**

- في المرات القادمة، بدلاً من استخدام متصفح الويب(RapidTables) ، يُنصح باستخدام سطر الأوامر مباشرة للسرعة.
- أمر مقترح python3 -c "print(bin(48))" *:يعطي النتيجة فوراً دون مغادرة التيرمينال.*

3. **فحص الكود :**يُفضل دائماً قراءة كود البايثون قبل تشغيله باستخدام أمر cat أو nano، للتأكد من أنه لا يحتوي على أوامر ضارة وللفهم الأعمق لكيفية عمل التحدي.